Lecture 9

Pete Manolios
Northeastern

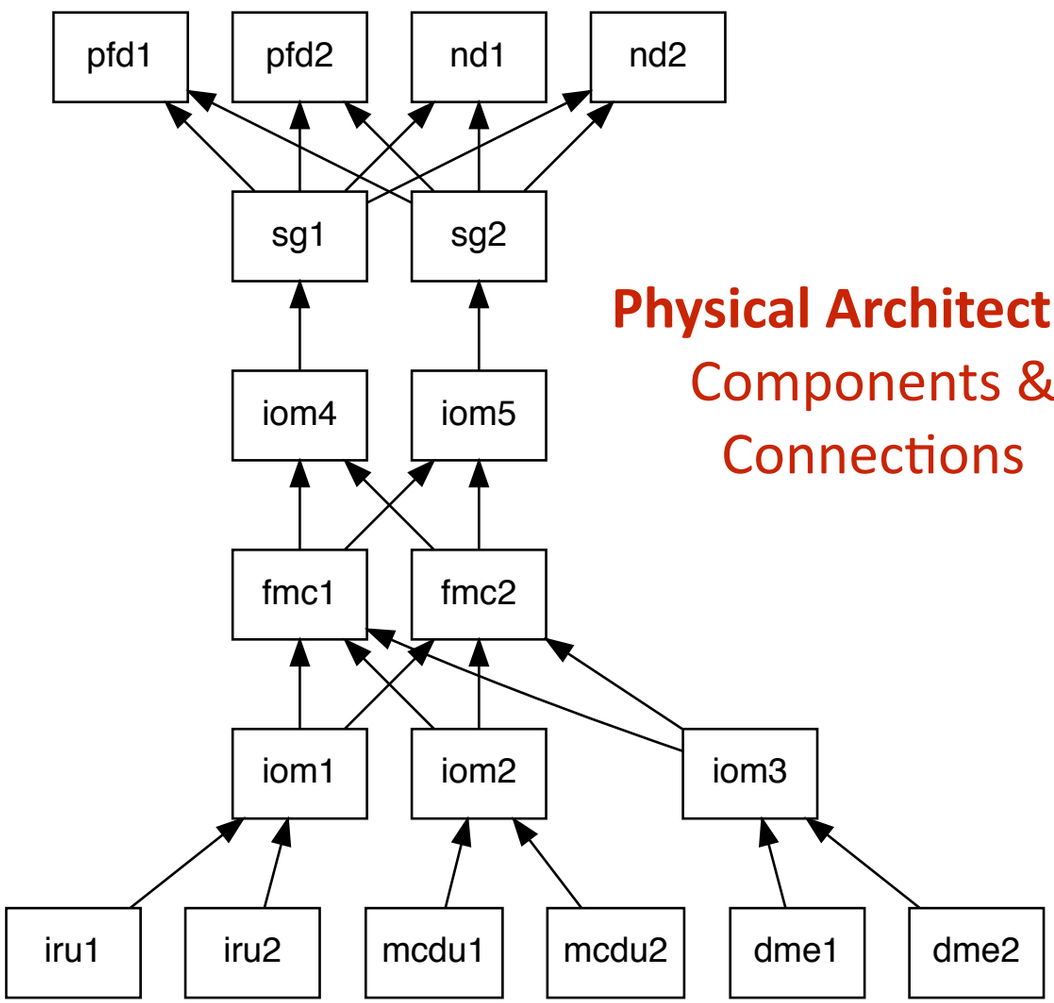
Computer-Aided Reasoning, Lecture 9

Safety Analysis Example

- The book has many interesting applications of SAT, e.g., circuits
- Let's look at one example in more depth which highlights some interesting issues: modeling, formula simplification, probability.
- See https://gitlab.com/pmanolios/safety-analysis
- Look at Webpage

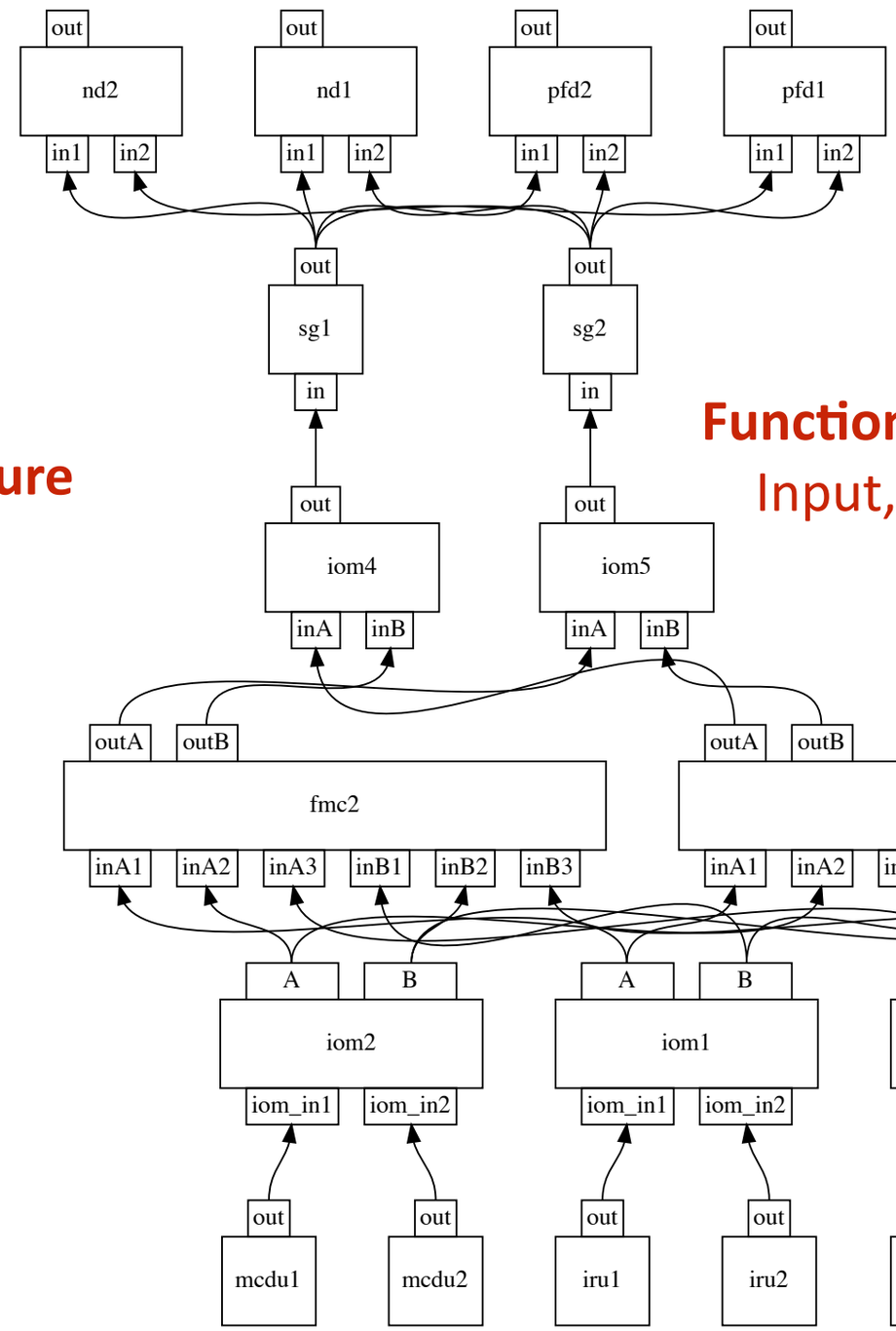
777-Inspired Architecture

Modeling loss of display of navigation information



Physical Architecture

Components & Connections

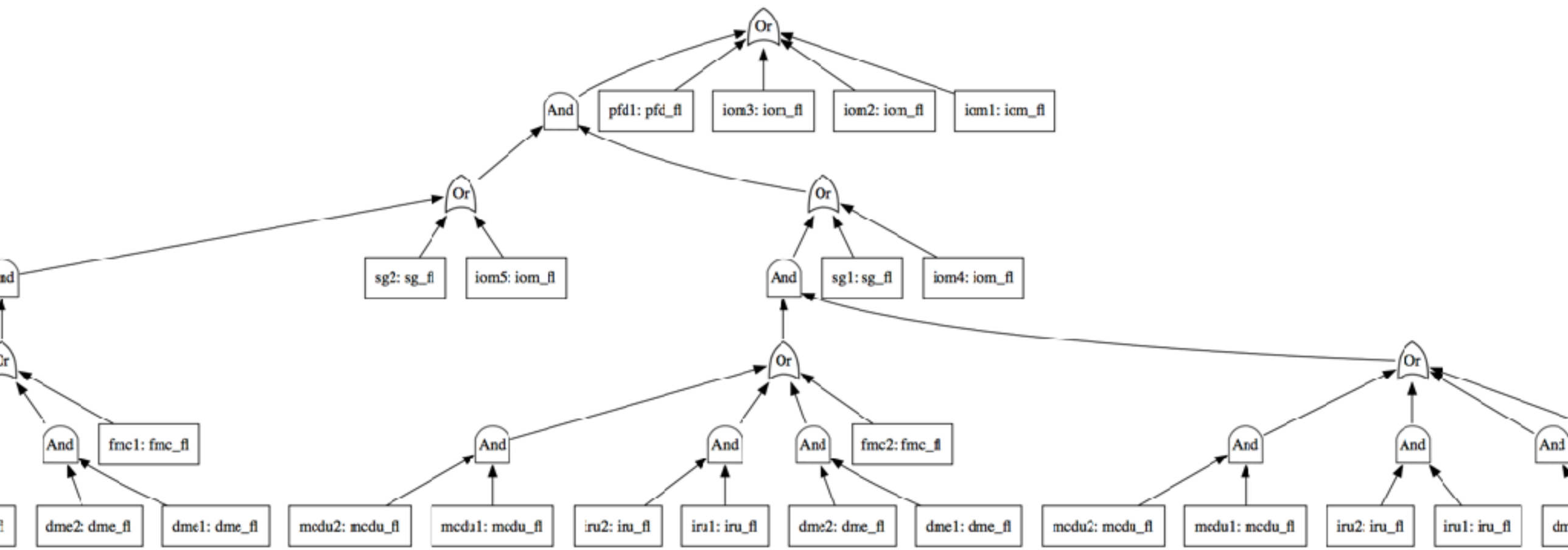


Functional Architecture

Input, Output Ports

Automated Fault Tree Synthesis

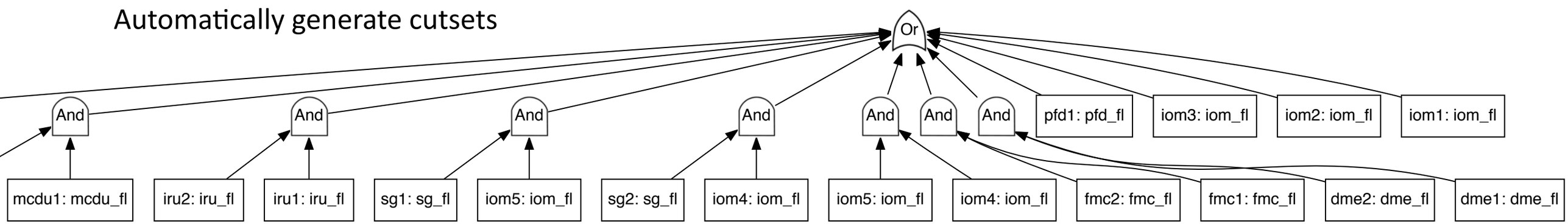
Fault trees automatically synthesized from models
Model update, reuse, maintenance, verification, analysis capabilities



Architectural Analysis

Quantitative analysis:
Top level fault probability 3.0002e-06

Qualitative analysis:
Automatically generate cutsets



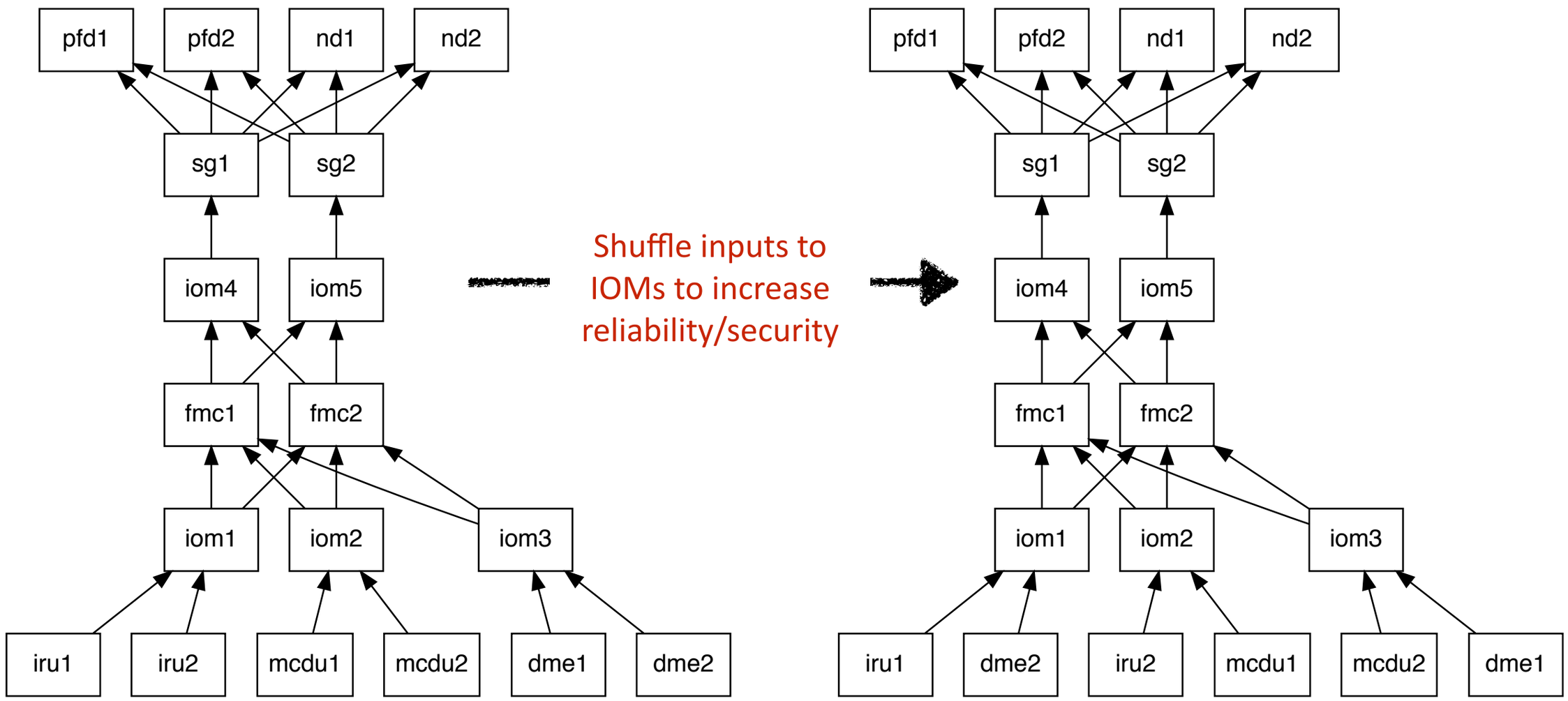
Automatically generate importance measures

Components	Probability	Importance Measure
iom1:iom_fl	9.9999E-07	0.3333
iom2:iom_fl	9.9999E-07	0.3333
iom3:iom_fl	9.9999E-07	0.3333
pfd1:pfd_fl	2.0000E-10	6.6662E-05
dme1:dme_fl; dme2:dme_fl	9.9999E-13	3.3331E-07

. . .

IOMs contribute most to top-level probability. **Remove the single points of failure.**

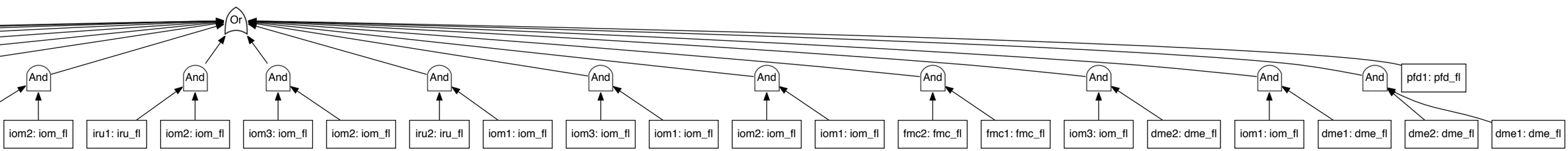
Architectural Trades/Synthesis



Architectural Analysis

Quantitative analysis:
Top level fault probability 2.1599e-10 (before: 3.0002e-06)

Qualitative analysis:
Automatically generate cutsets



Automatically generate importance measures

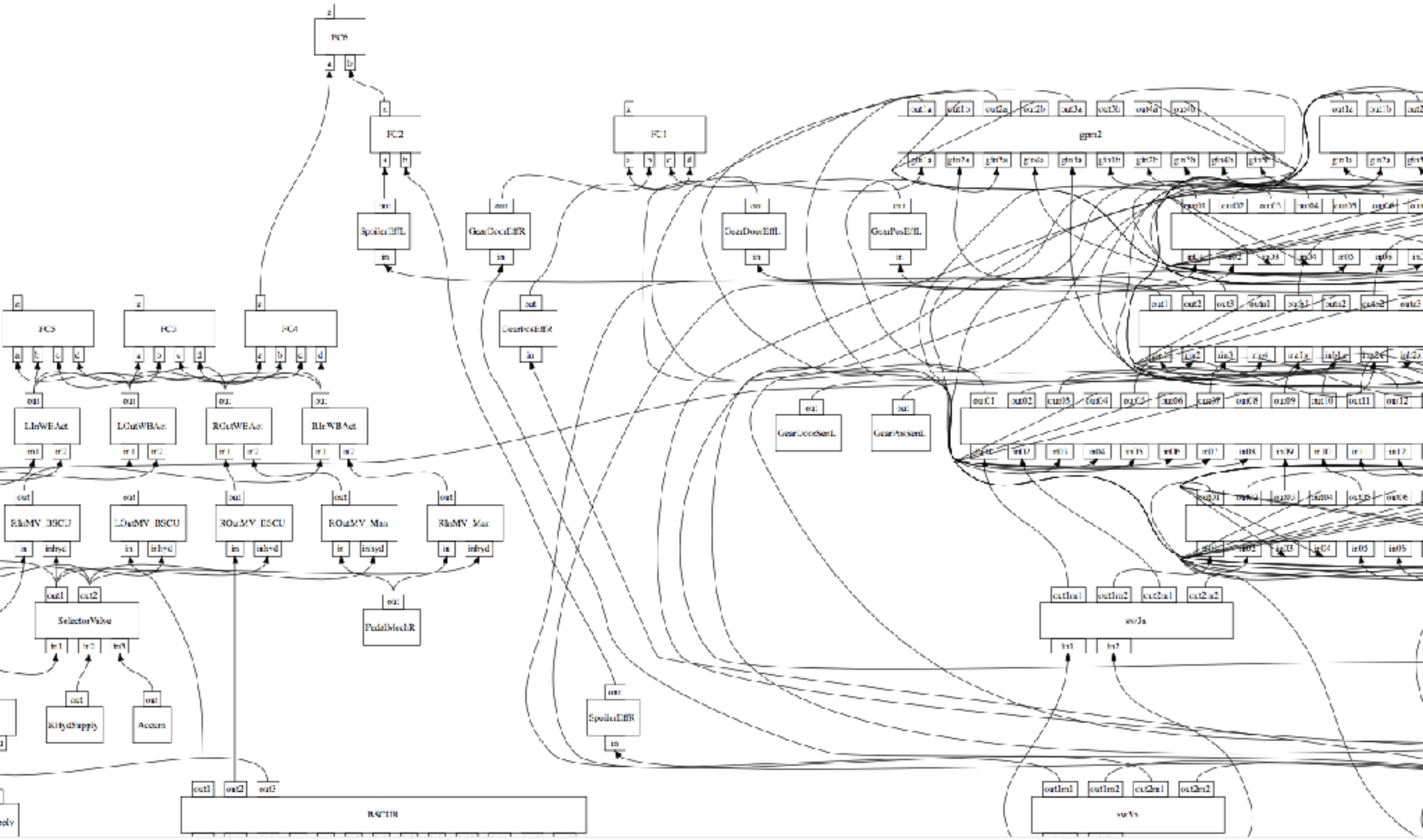
Components	Probability	Importance Measure
pfd1:pfd_fl	2.0000E-10	0.9259
dme1:dme_fl; dme2:dme_fl	9.9999E-13	0.0046
dme1:dme_fl; iom1:iom_fl	9.9999E-13	0.0046
iom1:iom_fl; iom3:iom_fl	9.9999E-13	0.0046
iru1:iru_fl; iru2:iru_fl	9.9999E-13	0.0046

IOMs are no longer single points of failure. **You need to compromise at least one other system for a failure.**

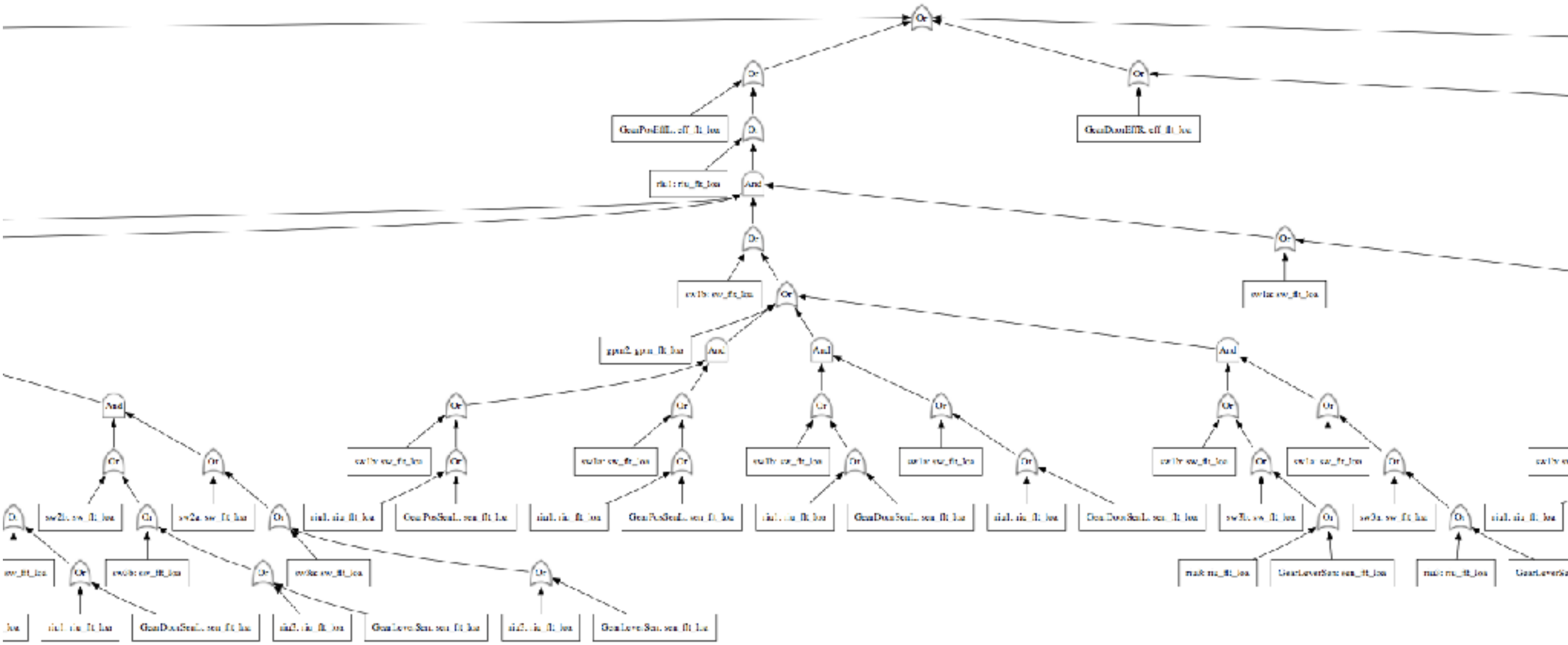
. . .

Slides by Pete Manolios for CS4820

Functional Architecture



Fault Tree Synthesized

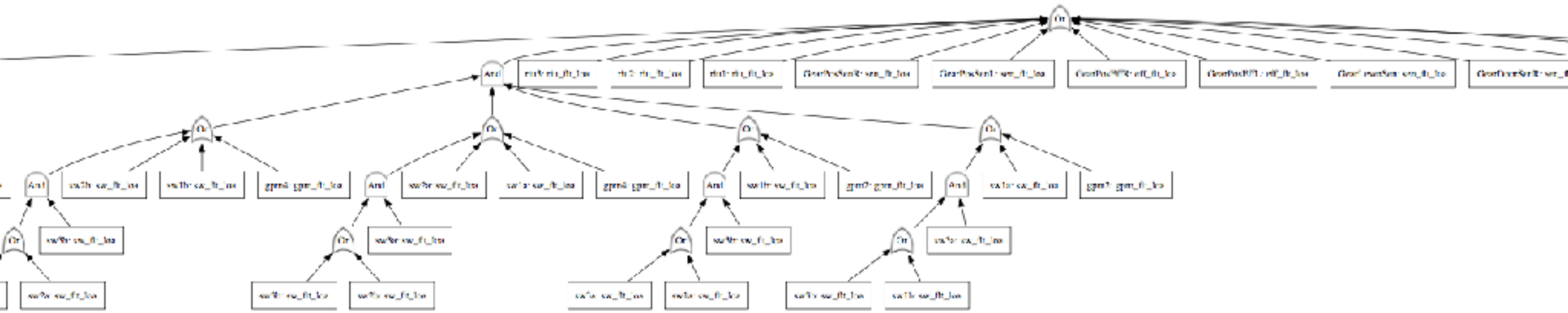


Large fault tree: 549 nodes, 261 Gates, Depth 11

The problem: safety engineer has to perform mental gymnastics to understand

The challenge: can we generate fault trees that a human might generate?

Fault Tree Reduction Engine



Nodes: 549 ➔ 83, Gates: 261 ➔ 27, Depth: Depth 11 ➔ 5

Notes: reduction engine does not increase depth

returns semantically equivalent tree

yields significant simplification of fault tree

fault trees now look like something a human generated

DP SAT Algorithm

- Input: CNF formula, Output: SAT/UNSAT
- Base case: empty clause: UNSAT
- Base case: no clauses: SAT
  - Apply these two rules until fixpoint
    - Pure literal rule
    - BCP
  - Choose var, say *x*, perform all possible resolutions, remove trivial clauses and clauses containing *x*
  - Repeat
- Existentially quantify variables, one at a time
- Problem: space blow-up

DPLL SAT Algorithm

- Boolean Constraint Propagation (BCP)
- Base case: empty clause: UNSAT
- Remove clauses containing pure literals (modern solvers don't do this)
- Base case: no clauses: SAT
- Choose some var, say x
  (if removing pure literals, x has to appear in both phases)
  - Add {x} and recursively call DPLL
  - Add {¬x} and recursively call DPLL
  - If one of the calls returns SAT, return SAT
  - Else return UNSAT
- Correctness follows from Shannon expansion!
- In contrast to DP, space is not a problem

DPLL SAT Example

Input Clauses

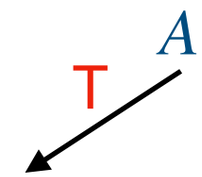
1. $\{A, B\}$
2. $\{B, C\}$
3. $\{\neg B, C, D\}$
4. $\{\neg A, \neg X, Y\}$
5. $\{\neg A, X, Z\}$
6. $\{\neg A, \neg Y, Z\}$
7. $\{\neg A, X, \neg Z\}$
8. $\{\neg A, \neg Y, \neg Z\}$

- We are not using the pure literal rule in the example

DPLL SAT Example

Input Clauses

1. $\{A, B\}$	1.
2. $\{B, C\}$	2. $\{B, C\}$
3. $\{\neg B, C, D\}$	3. $\{\neg B, C, D\}$
4. $\{\neg A, \neg X, Y\}$	4. $\{\neg X, Y\}$
5. $\{\neg A, X, Z\}$	5. $\{X, Z\}$
6. $\{\neg A, \neg Y, Z\}$	6. $\{\neg Y, Z\}$
7. $\{\neg A, X, \neg Z\}$	7. $\{X, \neg Z\}$
8. $\{\neg A, \neg Y, \neg Z\}$	8. $\{\neg Y, \neg Z\}$

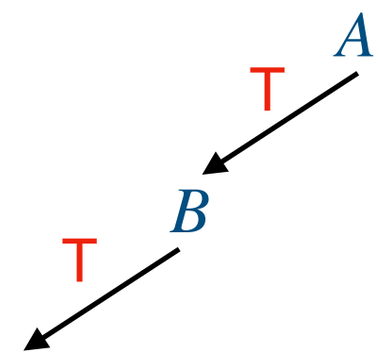


- We are not using the pure literal rule in the example

DPLL SAT Example

Input Clauses

1. $\{A, B\}$	1.
2. $\{B, C\}$	2.
3. $\{\neg B, C, D\}$	3. $\{C, D\}$
4. $\{\neg A, \neg X, Y\}$	4. $\{\neg X, Y\}$
5. $\{\neg A, X, Z\}$	5. $\{X, Z\}$
6. $\{\neg A, \neg Y, Z\}$	6. $\{\neg Y, Z\}$
7. $\{\neg A, X, \neg Z\}$	7. $\{X, \neg Z\}$
8. $\{\neg A, \neg Y, \neg Z\}$	8. $\{\neg Y, \neg Z\}$

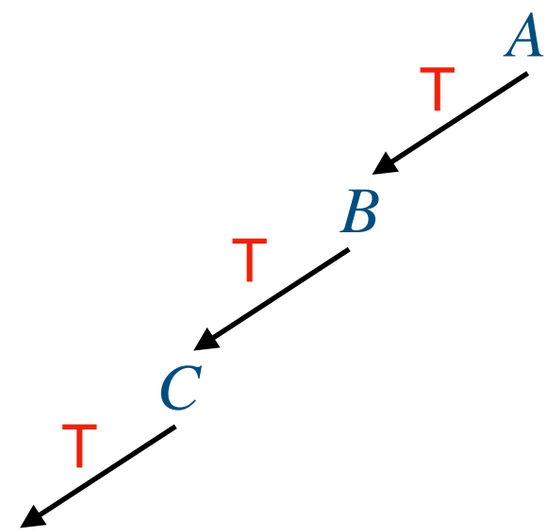


- We are not using the pure literal rule in the example

DPLL SAT Example

Input Clauses

1. $\{A, B\}$	1.
2. $\{B, C\}$	2.
3. $\{\neg B, C, D\}$	3.
4. $\{\neg A, \neg X, Y\}$	4. $\{\neg X, Y\}$
5. $\{\neg A, X, Z\}$	5. $\{X, Z\}$
6. $\{\neg A, \neg Y, Z\}$	6. $\{\neg Y, Z\}$
7. $\{\neg A, X, \neg Z\}$	7. $\{X, \neg Z\}$
8. $\{\neg A, \neg Y, \neg Z\}$	8. $\{\neg Y, \neg Z\}$

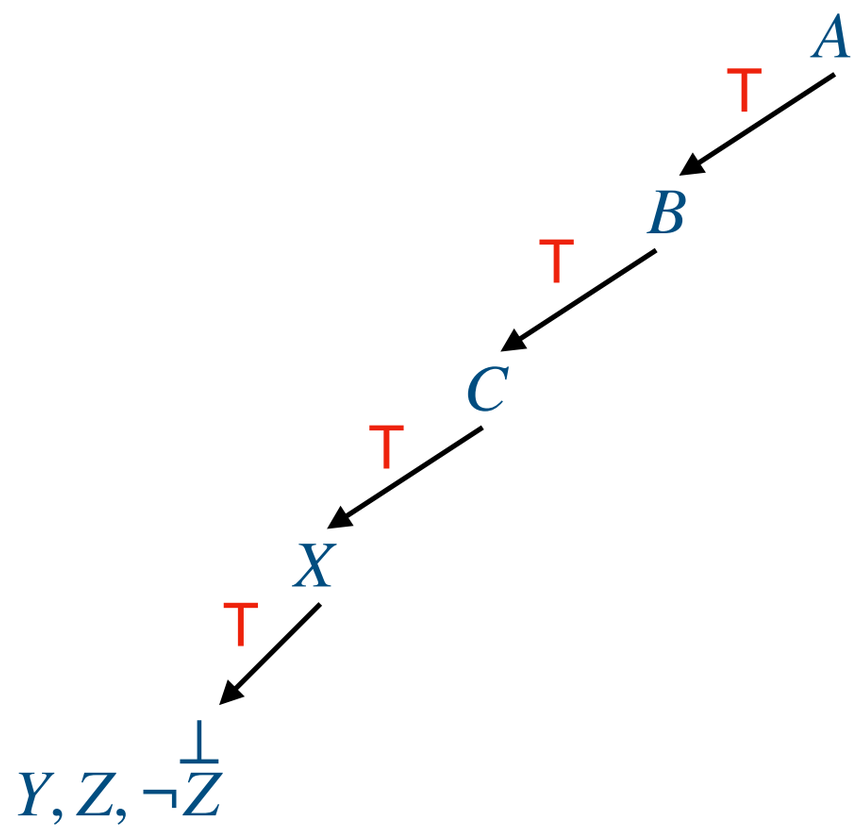


- We are not using the pure literal rule in the example

DPLL SAT Example

Input Clauses

1. $\{A, B\}$	1.
2. $\{B, C\}$	2.
3. $\{\neg B, C, D\}$	3.
4. $\{\neg A, \neg X, Y\}$	4. $\{Y\}$
5. $\{\neg A, X, Z\}$	5.
6. $\{\neg A, \neg Y, Z\}$	6. $\{\neg Y, Z\}$
7. $\{\neg A, X, \neg Z\}$	7.
8. $\{\neg A, \neg Y, \neg Z\}$	8. $\{\neg Y, \neg Z\}$

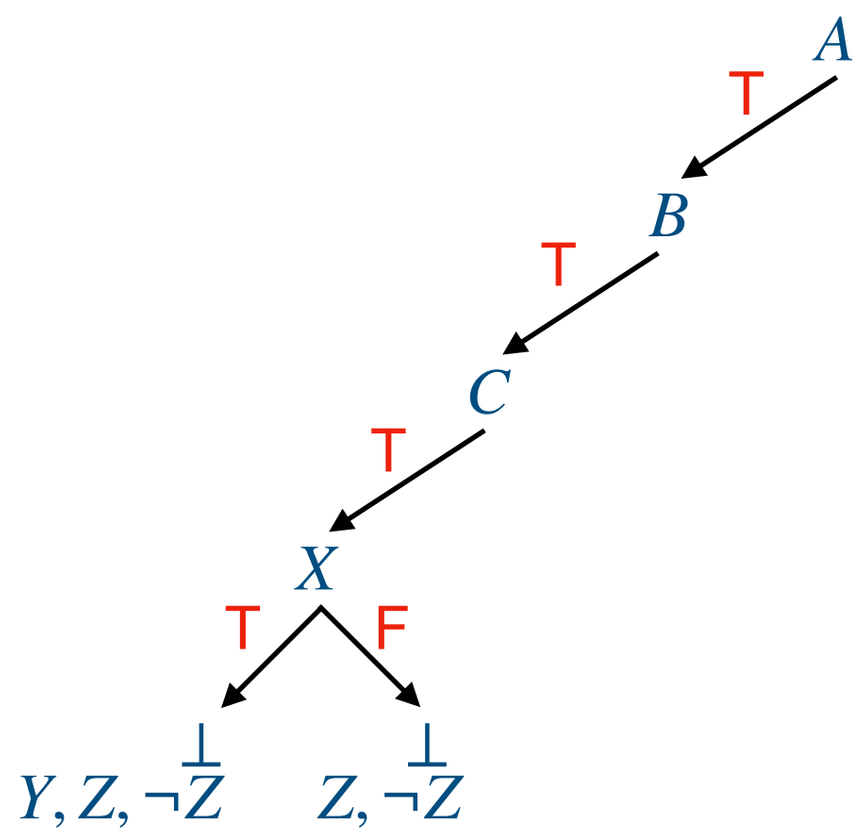


- We are not using the pure literal rule in the example

DPLL SAT Example

Input Clauses

1. $\{A, B\}$	1.
2. $\{B, C\}$	2.
3. $\{\neg B, C, D\}$	3.
4. $\{\neg A, \neg X, Y\}$	4.
5. $\{\neg A, X, Z\}$	5. $\{Z\}$
6. $\{\neg A, \neg Y, Z\}$	6. $\{\neg Y, Z\}$
7. $\{\neg A, X, \neg Z\}$	7. $\{\neg Z\}$
8. $\{\neg A, \neg Y, \neg Z\}$	8. $\{\neg Y, \neg Z\}$

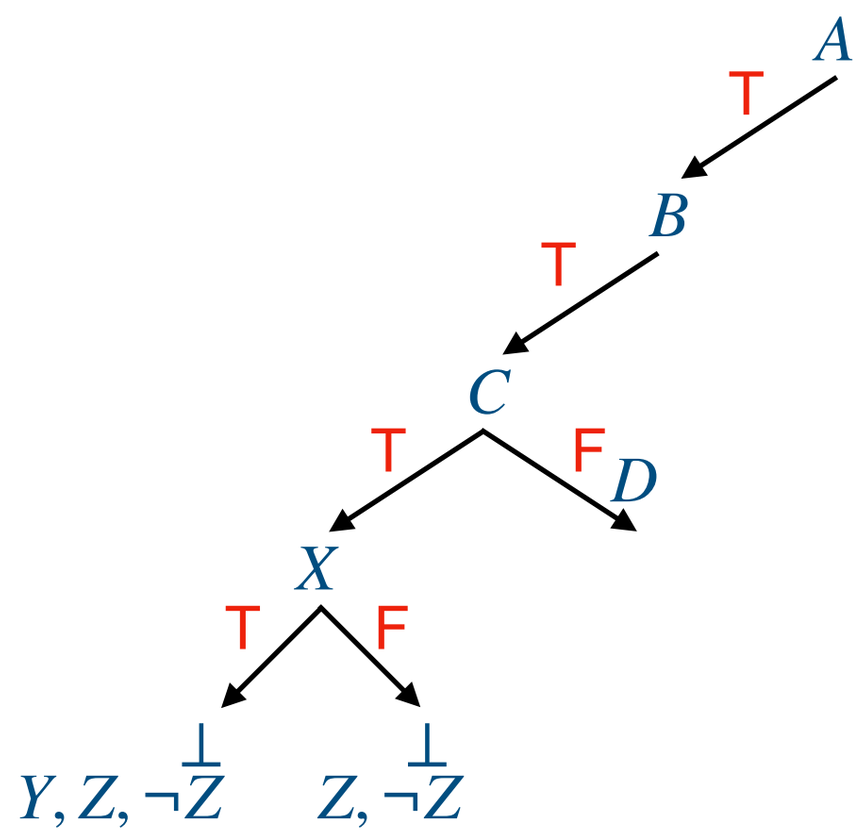


- We are not using the pure literal rule in the example
- Notice that when DPLL detects contradictions it backtracks chronologically

DPLL SAT Example

Input Clauses

1. $\{A, B\}$	1.
2. $\{B, C\}$	2.
3. $\{\neg B, C, D\}$	3. $\{D\}$
4. $\{\neg A, \neg X, Y\}$	4. $\{\neg X, Y\}$
5. $\{\neg A, X, Z\}$	5. $\{X, Z\}$
6. $\{\neg A, \neg Y, Z\}$	6. $\{\neg Y, Z\}$
7. $\{\neg A, X, \neg Z\}$	7. $\{X, \neg Z\}$
8. $\{\neg A, \neg Y, \neg Z\}$	8. $\{\neg Y, \neg Z\}$

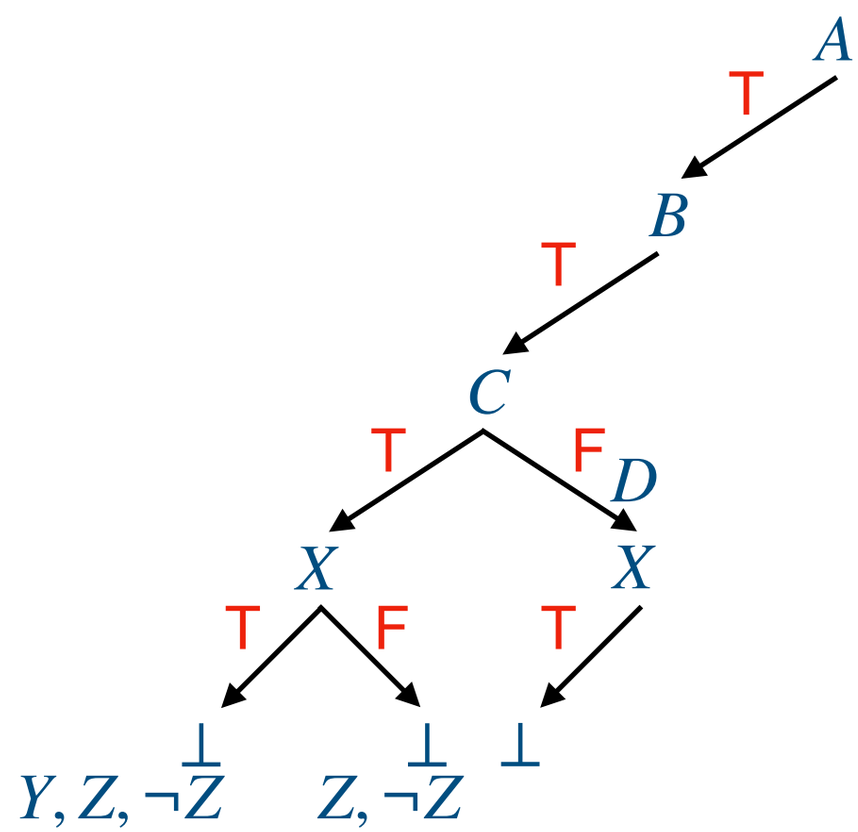


- We are not using the pure literal rule in the example
- Notice that when DPLL detects contradictions it backtracks chronologically

DPLL SAT Example

Input Clauses

1. $\{A, B\}$	1.
2. $\{B, C\}$	2.
3. $\{\neg B, C, D\}$	3. $\{D\}$
4. $\{\neg A, \neg X, Y\}$	4. $\{Y\}$
5. $\{\neg A, X, Z\}$	5.
6. $\{\neg A, \neg Y, Z\}$	6. $\{\neg Y, Z\}$
7. $\{\neg A, X, \neg Z\}$	7.
8. $\{\neg A, \neg Y, \neg Z\}$	8. $\{\neg Y, \neg Z\}$

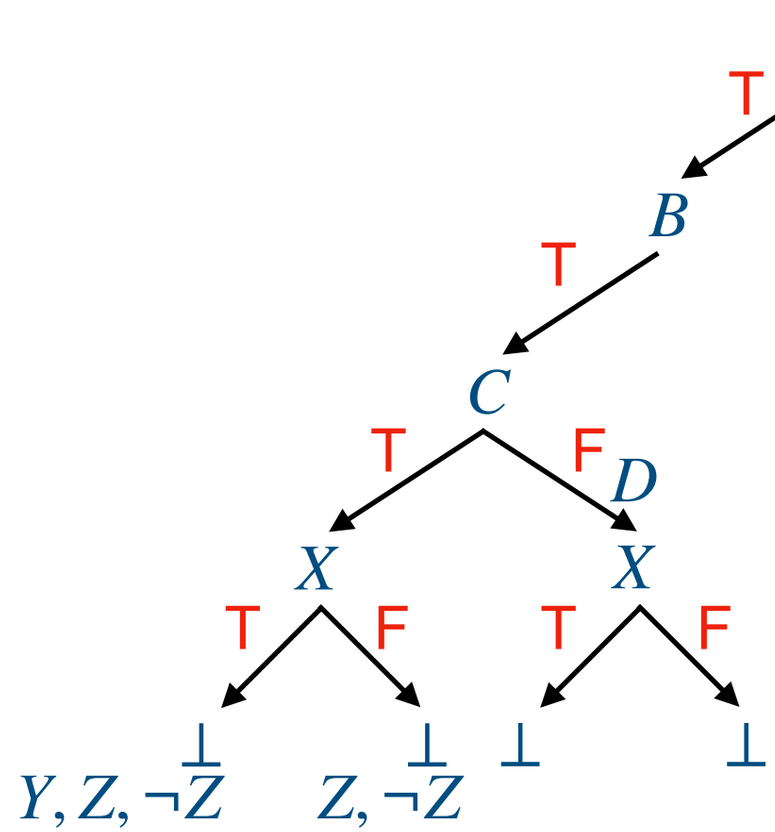


- We are not using the pure literal rule in the example
- Notice that when DPLL detects contradictions it backtracks chronologically

DPLL SAT Example

Input Clauses

1. $\{A, B\}$	1.
2. $\{B, C\}$	2.
3. $\{\neg B, C, D\}$	3. $\{D\}$
4. $\{\neg A, \neg X, Y\}$	4.
5. $\{\neg A, X, Z\}$	5. $\{Z\}$
6. $\{\neg A, \neg Y, Z\}$	6. $\{\neg Y, Z\}$
7. $\{\neg A, X, \neg Z\}$	7. $\{\neg Z\}$
8. $\{\neg A, \neg Y, \neg Z\}$	8. $\{\neg Y, \neg Z\}$

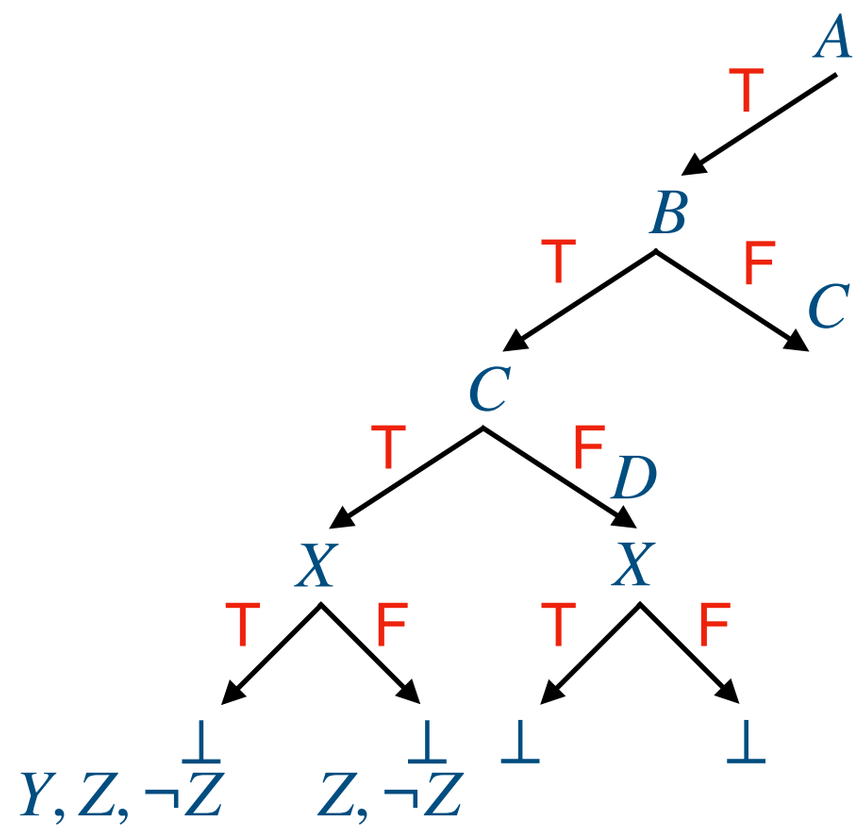


- We are not using the pure literal rule in the example
- Notice that when DPLL detects contradictions it backtracks chronologically

DPLL SAT Example

Input Clauses

1. $\{A, B\}$	1.
2. $\{B, C\}$	2. $\{C\}$
3. $\{\neg B, C, D\}$	3.
4. $\{\neg A, \neg X, Y\}$	4. $\{\neg X, Y\}$
5. $\{\neg A, X, Z\}$	5. $\{X, Z\}$
6. $\{\neg A, \neg Y, Z\}$	6. $\{\neg Y, Z\}$
7. $\{\neg A, X, \neg Z\}$	7. $\{X, \neg Z\}$
8. $\{\neg A, \neg Y, \neg Z\}$	8. $\{\neg Y, \neg Z\}$

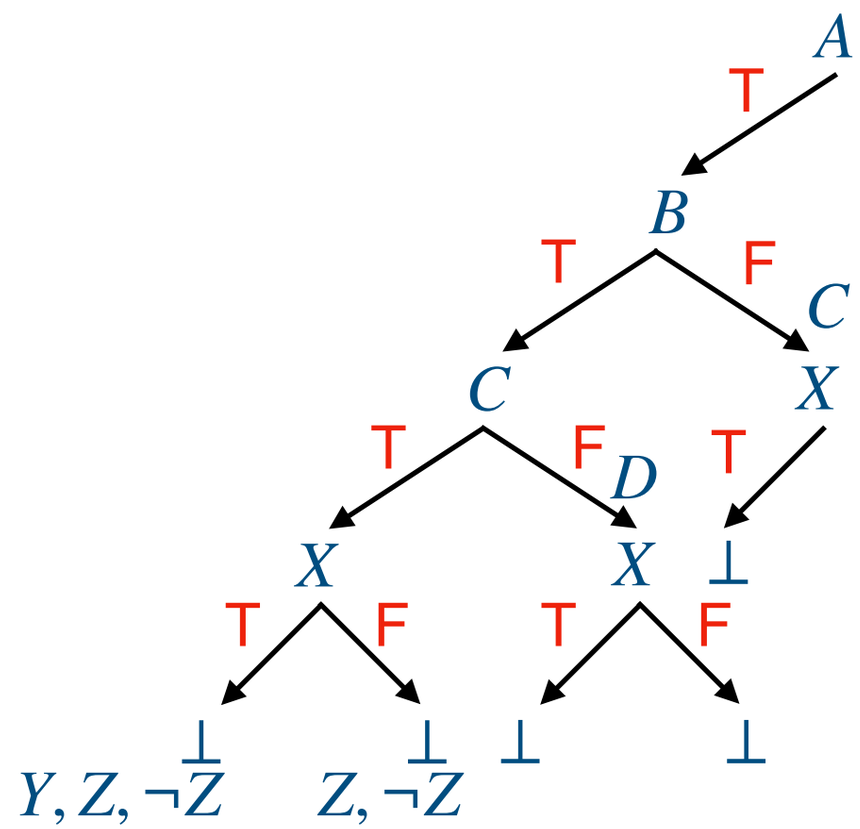


- We are not using the pure literal rule in the example
- Notice that when DPLL detects contradictions it backtracks chronologically
- When we get a contradiction with X, we try ¬X, then we go back and try ¬C and X, ¬X again, …

DPLL SAT Example

Input Clauses

1. $\{A, B\}$	1.
2. $\{B, C\}$	2. $\{C\}$
3. $\{\neg B, C, D\}$	3.
4. $\{\neg A, \neg X, Y\}$	4. $\{Y\}$
5. $\{\neg A, X, Z\}$	5.
6. $\{\neg A, \neg Y, Z\}$	6. $\{\neg Y, Z\}$
7. $\{\neg A, X, \neg Z\}$	7.
8. $\{\neg A, \neg Y, \neg Z\}$	8. $\{\neg Y, \neg Z\}$

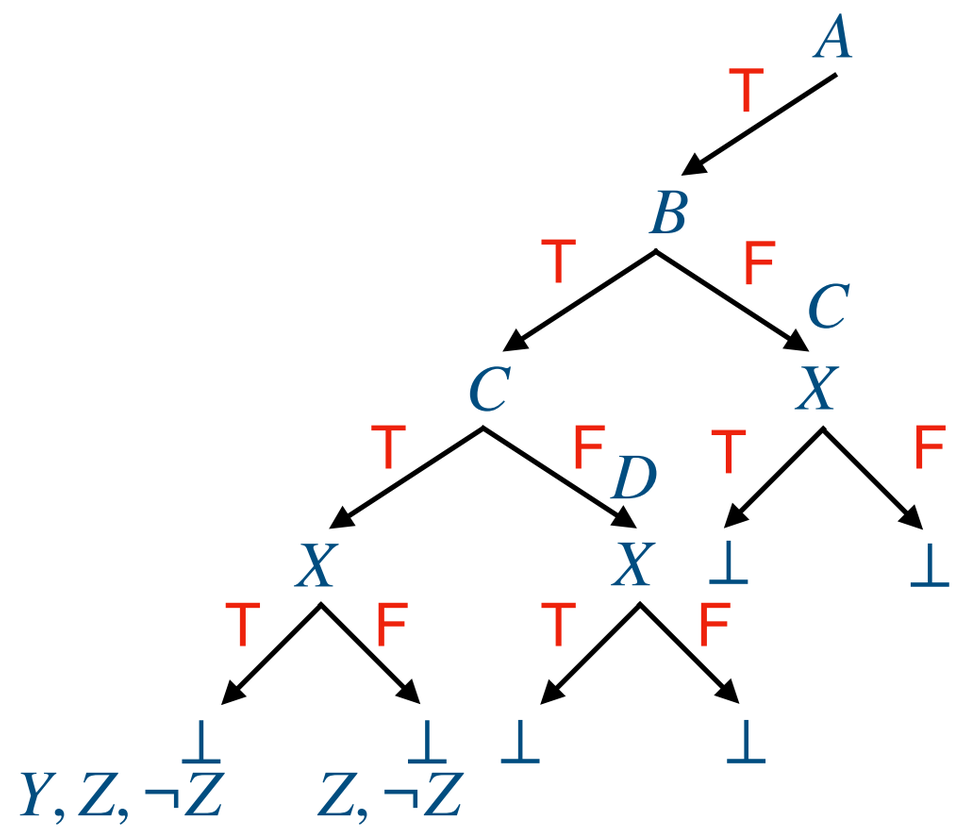


- We are not using the pure literal rule in the example
- Notice that when DPLL detects contradictions it backtracks chronologically
- When we get a contradiction with X, we try ¬X, then we go back and try ¬C and X, ¬X again, …

DPLL SAT Example

Input Clauses

1. $\{A, B\}$	1.
2. $\{B, C\}$	2. $\{C\}$
3. $\{\neg B, C, D\}$	3.
4. $\{\neg A, \neg X, Y\}$	4.
5. $\{\neg A, X, Z\}$	5. $\{Z\}$
6. $\{\neg A, \neg Y, Z\}$	6. $\{\neg Y, Z\}$
7. $\{\neg A, X, \neg Z\}$	7. $\{\neg Z\}$
8. $\{\neg A, \neg Y, \neg Z\}$	8. $\{\neg Y, \neg Z\}$

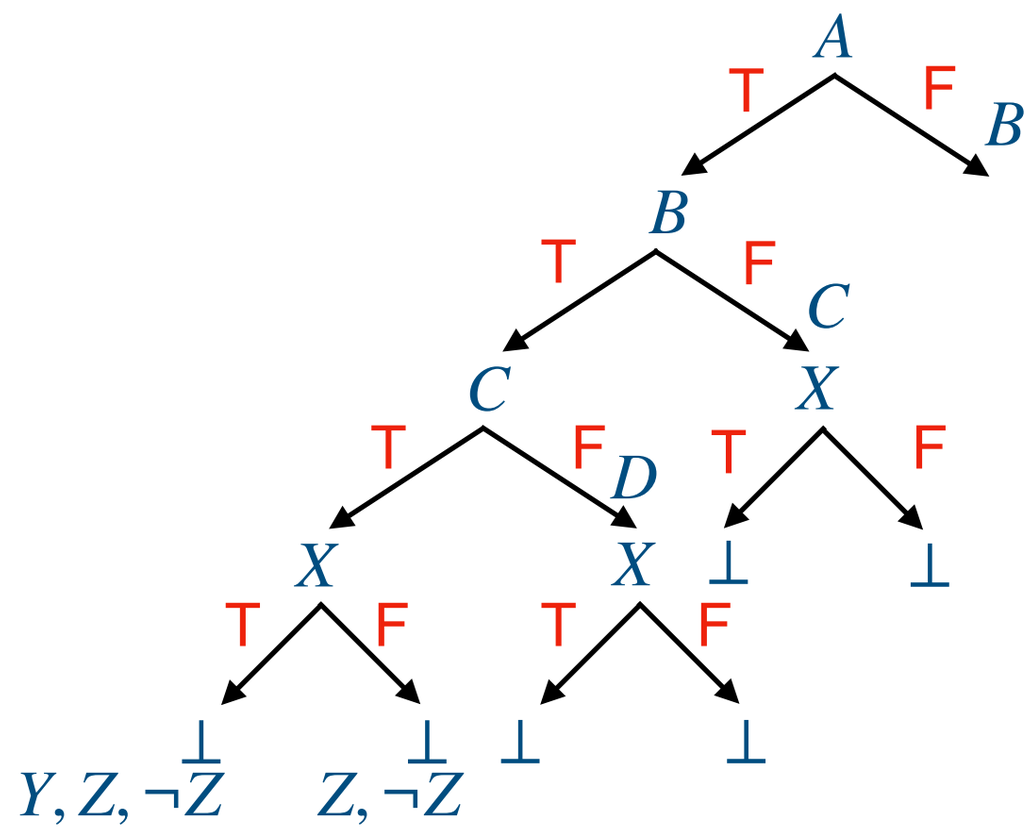


- We are not using the pure literal rule in the example
- Notice that when DPLL detects contradictions it backtracks chronologically
- When we get a contradiction with X, we try ¬X, then we go back and try ¬C and X, ¬X again, …

DPLL SAT Example

Input Clauses

1. $\{A, B\}$	1. $\{B\}$
2. $\{B, C\}$	2.
3. $\{\neg B, C, D\}$	3. $\{C, D\}$
4. $\{\neg A, \neg X, Y\}$	4.
5. $\{\neg A, X, Z\}$	5.
6. $\{\neg A, \neg Y, Z\}$	6.
7. $\{\neg A, X, \neg Z\}$	7.
8. $\{\neg A, \neg Y, \neg Z\}$	8.

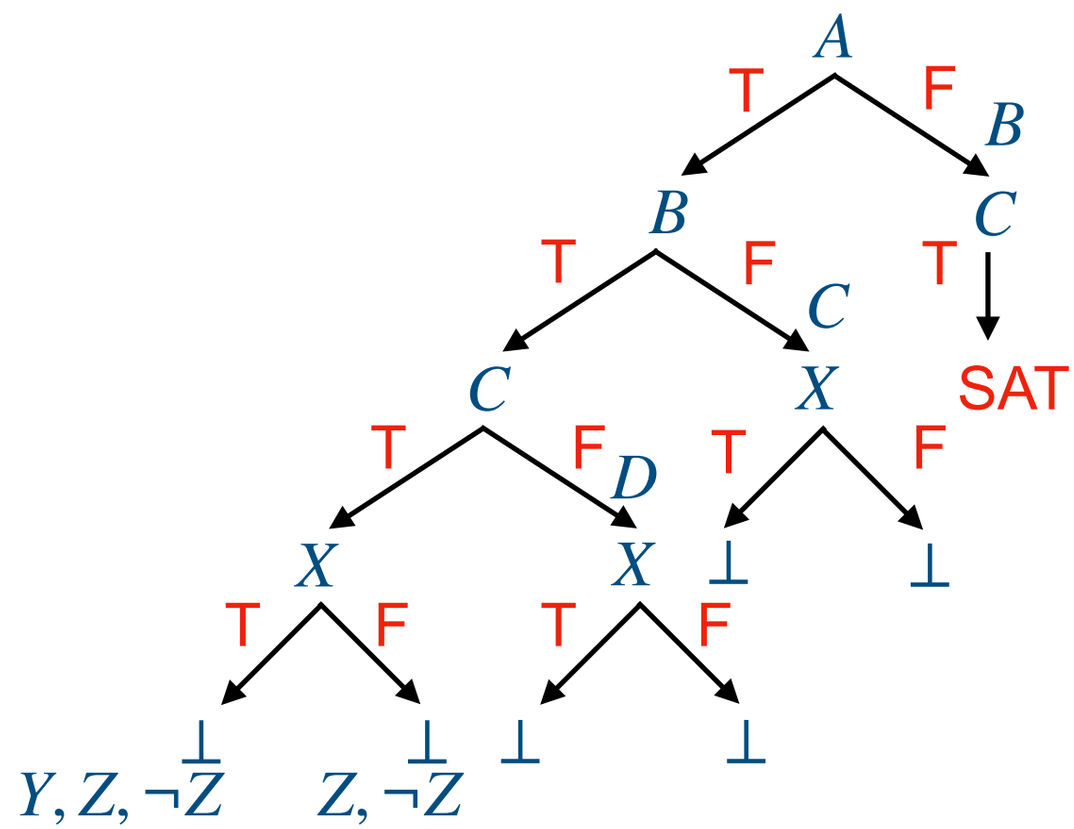


- We are not using the pure literal rule in the example
- Notice that when DPLL detects contradictions it backtracks chronologically
- When we get a contradiction with X, we try ¬X, then we go back and try ¬C and X, ¬X again, …
- Finally, we realize A must be *false,* which implies *B* is *true* (BCP), …

DPLL SAT Example

Input Clauses

1. $\{A, B\}$	1. $\{B\}$
2. $\{B, C\}$	2.
3. $\{\neg B, C, D\}$	3.
4. $\{\neg A, \neg X, Y\}$	4.
5. $\{\neg A, X, Z\}$	5.
6. $\{\neg A, \neg Y, Z\}$	6.
7. $\{\neg A, X, \neg Z\}$	7.
8. $\{\neg A, \neg Y, \neg Z\}$	8.



- We are not using the pure literal rule in the example
- Notice that when DPLL detects contradictions it backtracks chronologically
- When we get a contradiction with X, we try ¬X, then we go back and try ¬C and X, ¬X again, …
- Finally, we realize A must be *false,* which implies *B* is *true* (BCP), …
- Can we avoid *this kind of* repeated search (can be exponential)?
  - Yes: non-chronological backtracking, clause learning, and many other major improvements

Modern CDCL Solvers

- Based on DPLL, but with conflict-driven clause learning
- Data structures to speed up BCP: 2-watched literal scheme
- Data structures for clause learning
- Decision heuristics: select recently active literals (VSIDS)
- Preprocessing: greedy variable elimination
- Inprocessing: interleave preprocessing & search
- Clause deletion: learned clauses lead to memory & efficiency problems, so delete large, inactive clauses
- Random restarts: keep learned clauses, but restart
  - avoids getting stuck in hard part of search space
  - phase saving: pick last phase of assignment

DIMACS Format

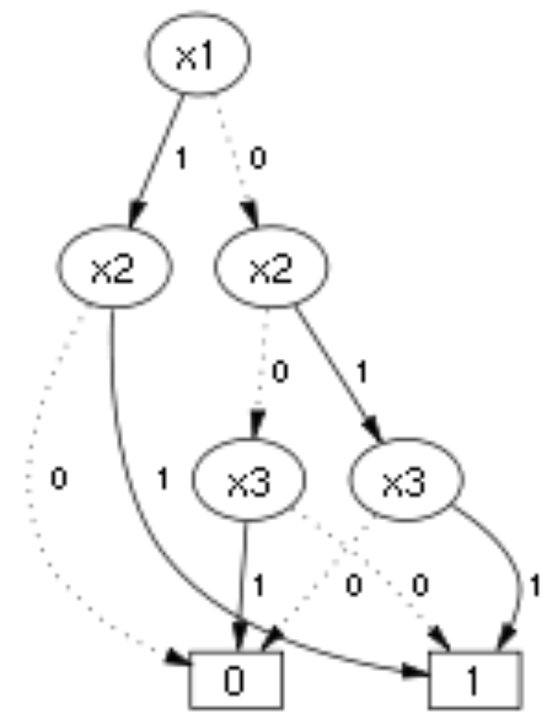
- Modern SAT solvers accept input in CNF
  - Dimacs format:
  - 1 -3 4 5 0
  - 2 -4 7 0
  - …

Normal Forms

- Minimizing DNF has many applications
  - this is used to analyze the reliability of safety-critical systems
- CNF is the input format of modern SAT solvers
  - this is the so-called DIMACS format
  - modern SAT solvers can solve industrial problems with 1M variables
- There are many other “normal” forms for Boolean formulae
  - decision trees: widely used in machine learning
  - BDDs: very powerful representation used in verification, AI, program analysis, …

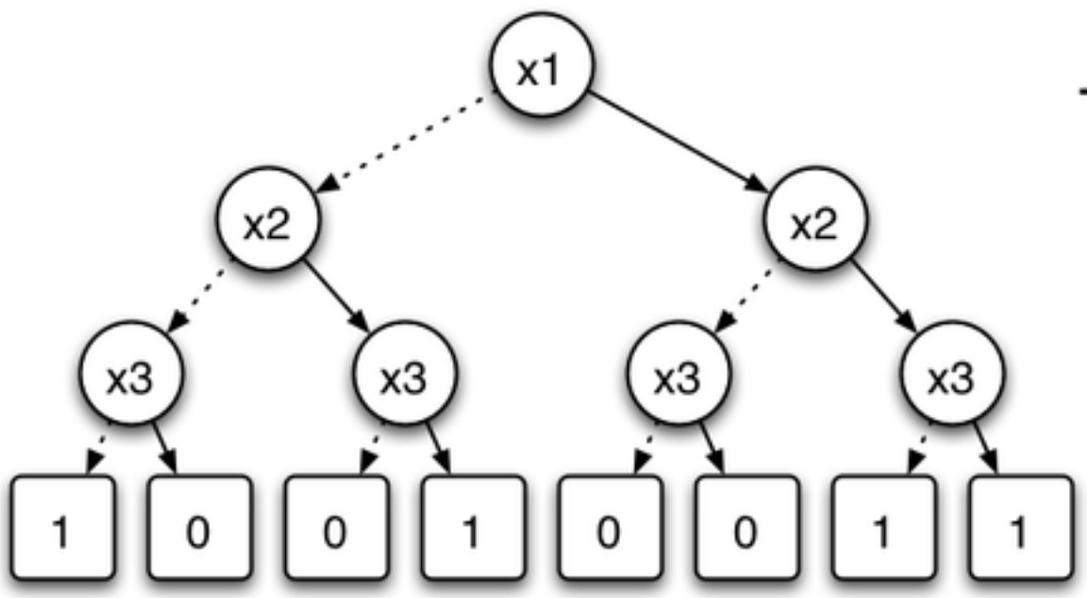
BDDs and Decision Trees

- A BDD on $x_1, \ldots, x_n$ is a DAG G=(V, E) where
  - exactly 1 vertex has indegree 0 (the root)
  - all vertices have outdegree 0 (leaves) or 2 (inner nodes)
  - the inner nodes are labeled from $\{x_1, \ldots, x_n\}$
  - the leaves are labeled from {0, 1}
  - one of the edges from an inner node is labeled by 0; the other by 1
- The BDD G=(V, E) represents a Boolean function, say f
  - for any assignment A in B^n, f(A) is computed recursively from root
  - if we reach a leaf, return the label
  - for inner nodes, say labeled with x_i, take the edge labeled by $A(x_i)$
- A decision tree is a BDD whose graph is a tree
- A BDD is an OBDD if there is a permutation on p={1,2, …, n} s.t. for all edges (u, v) in E, where u, v are labeled by x_i, x_j, we have that $p_i < p_j$
- An OBDD is an ROBDD if it has no isomorphic subgraphs and all children are distinct

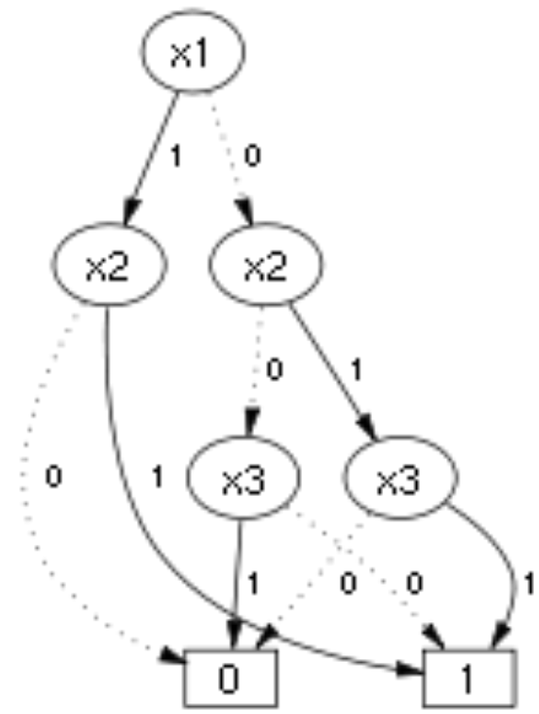


Images from Wikipedia

BDDs and Decision Trees



x1	x2	x3	f
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



Decision Tree for f

ROBDD for f

How do we generate DNF from a decision tree? ROBDD?

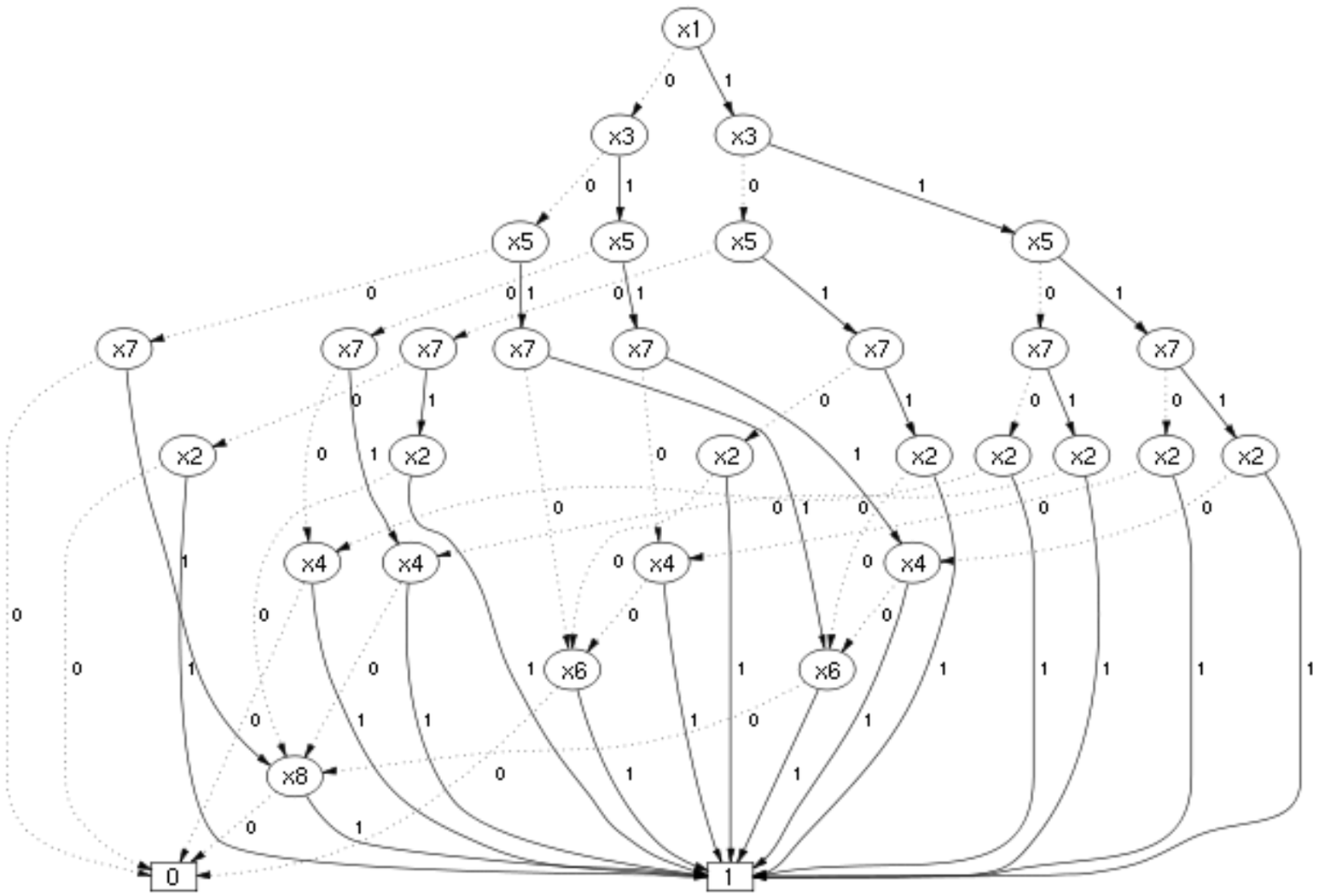
Images from Wikipedia

BDDs

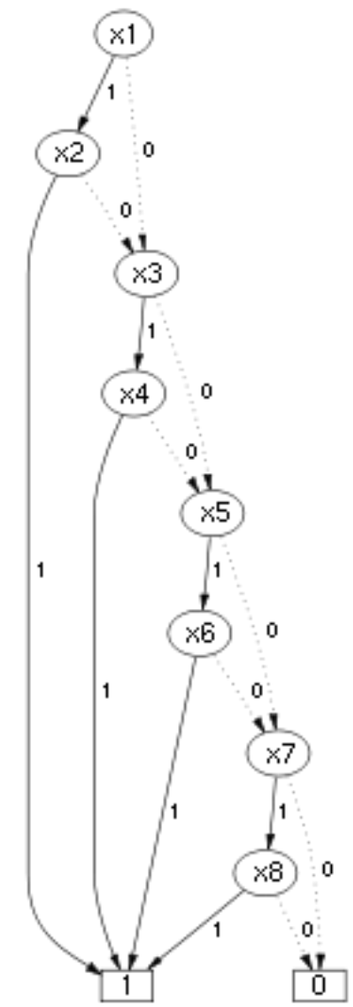
- Decision trees are widely used, e.g., in machine learning (ID3, C4.5, …)
- BDDs are widely used (BDD usually means ROBDD)
  - Popularized by Bryant
  - Very efficient algorithms for constructing, manipulating BDDs
  - Used in verification, synthesis, fault trees, security, AI, model checking, static analysis, …
  - Bryant's paper was the most cited research paper (at some point)
  - Many BDD packages available
- Once a variable ordering is selected, BDDs are canonical!
  - Construct decision tree using Shannon expansion and merge isomorphic nodes, remove nodes who children are equal until you reach a fixpoint
  - To see, this note that BDDs are essentially DFAs that recognize strings in $\{0,1\}^n$ and such automata can be minimized (note nodes with equal children remain)
  - So, checking equality is just pointer equality (with appropriate data structures)
  - Can be used for model checking: represent set of reachable states & transition system with BDDs

Variable Ordering for BDDs

Variable ordering matters: finding the best ordering is hard.



Bad Ordering



Good Ordering

Images from Wikipedia

SAT Solving Algorithms

- Symbolic SAT solving
  - Use BDDs, but try hard to not get blowup of intermediate BDDs
  - So, existential quantification and other techniques are used
  - The goal is to minimize the size of intermediate BDDs
- SAT by inference rules
- Stalmarck's Algorithm
  - Preprocess the formula
  - Apply simple inference rules: 0-saturation
  - Apply dilemma rule: for each variable x, 0-saturate $f|\ell$ and $f|\neg\ell$, and all common conclusions to f. Repeat until fixpoint: 1-saturation
  - n-saturation: case split over all combinations on n variables